Migrating a Legacy Modules Without Regression and critically reviewing AI-Generated Code

Objective

By the end of this training, you'll know how to tackle one of the trickiest jobs in engineering: migrating a legacy system, without breaking whatever it is people are secretly relying on. You'll learn to use AI to speed up the grunt work mapping out hidden dependencies, translating old code but the mindset matters more than the tooling. Every AI suggestion, every migration plan it hands you, is a starting point to poke holes in, not something to trust and ship. Your job is to hold the line: verify it against a Golden Master, red-team it, and only let it through once you've actually proven nothing changed.

Outline

- Understanding the Migration Task: Pinning Down the Ambiguity
- Using AI to Map Undocumented Legacy Dependencies
- Building a Safety Net Before Changing Anything
- AI-Assisted Migration and Optimization Strategies
- Verifying No Regression
- Handing Off to Deployment
- Red-Teaming the AI-Generated Result
- A Practical Red-Team Checklist
- Prompting Practices That Reduce Bad Output
- Bringing It Together

Introduction

Every team has that one system old, half-understood, and quietly running something important. Nobody wants to touch it, but eventually somebody has to migrate it.

AI can genuinely help here. It reads code fast, spots patterns, and can translate old logic in seconds. But speed was never the hard part of migration judgment. AI doesn't know that an "ugly" bit of code was actually an intentional fix, and it doesn't know that a quirky old behavior was actually a bug nobody caught. It just sees what's there and runs with it, confidently, either way.

This module is about using AI to move faster without letting it make the calls that actually matter. You'll learn how to get real value out of it mapping dependencies, drafting tests, translating code while treating everything it produces as something to check, not trust. It's a fast pair of hands, not a decision-maker. That part's still on you.

Understanding the Migration Task

Migrating legacy systems is rarely a simple lift-and-shift operation. It is a complex engineering challenge that requires deep archaeology into undocumented decisions, unwritten business rules, and accidental features that users have come to rely on over years of operation.

To successfully navigate a legacy migration today, an engineer must act as both an investigator and a skeptical reviewer. You have to reverse-engineer the actual behavior of the legacy code, write robust

characterization tests to lock down that behavior, and then critically evaluate every line of AI-generated migration code against expectations.

What "migration" means in a JIRA ticket

In practice, migration tickets usually collapse one or more of these distinct engineering efforts into a single vague sentence:

Platform/Runtime Migration: Moving to a new language version, framework, or ORM with behavior held absolutely constant (zero behavioral change).

Structural Migration: Preserving behavior but reorganizing code (e.g., carving a service out of a monolith, moving from callbacks to async/await).

Optimization Migration: The ticket says "migrate," but the underlying task is performance. It permits changed internals (reducing latency, memory, or queries) as long as observable outputs match.

Behavioral Migration in Disguise: The ticket claims "just a migration" but smuggles in a bug fix or feature change ("while we're in there, also fix the rounding issue").

Ownership vs. Deployment Responsibility

A migration engineer is not always the deployment owner. Conflating the two roles causes two common failure modes:

Under-scoping the handoff: If you assume "my job ends when tests pass," you skip externalizing the context only you possess: what changed under the hood, edge cases you found, what you're unsure about, and how to roll back safely.
Over-scoping the change: It's tempting to "improve while you're in there" , upgrade a dependency or fix a nearby style issue. This is scope creep. AI assistants are highly prone to this; asked to migrate a function, they will often silently "fix" adjacent code without being asked.

Legacy Code as an Unreliable Narrator

Legacy code tells you what it does, not what it was meant to do. Comments go stale, variable names drift from their true purpose, and tests if they even exist often just encode the same misunderstandings as the code itself. A migration built purely by observing current behavior, especially with AI, risks two massive, contrasting failures:

Failure 1: Faithfully reproducing an accidental bug:
AI tools are literal. If they see a pattern, they assume it’s intentional.
Example: A legacy API uses LIMIT and OFFSET for pagination but forgets the ORDER BY clause. On the old database, it happened to return rows in insertion order, so nobody noticed. The AI migrates this exact query to a new distributed database. Without an explicit ORDER BY, the new database returns rows in random order, breaking pagination completely. The AI faithfully copied a bug that was previously hidden by infrastructure.

Failure 2: "Fixing" an intentional workaround:
AI likes idiomatic code and will gladly "clean up" ugly logic, missing that the ugliness was put there on purpose.
Example: An API controller catches a validation error but still returns an HTTP 200 OK. The AI correctly flags this as bad REST design and rewrites it to return a 400 Bad Request. What the code didn't say is that legacy mobile clients will hard-crash if they receive a 4xx error on this specific endpoint. By cleaning up an intentional workaround, the AI just broke the app for older clients.
This is the fundamental tension of migration: You are trying to capture actual behavior precisely enough to know if anything changed, without accidentally cementing bugs or breaking hidden fixes.

Using AI to Map Undocumented Legacy Dependencies

The goal of this phase is to move away from vague prompts ("explain this module") toward prompts that produce a structured, checkable artifact. A generic read-through prompt will produce something that reads well but hides critical gaps.

Techniques for Prompting AI to Trace the Unknown

A few specific patterns are worth adopting explicitly when tracing legacy code:

- **Ask for structure, not prose:** A prompt like "List every function this module calls, every function that calls into it, and every piece of state it reads/writes as a table" produces something reviewable. A generic "explain this" prompt produces a smooth summary that glides right over the hard parts.
- **Trace one axis at a time:** Don't bundle your requests. Use separate prompts for Call Chains (what calls X, what does X call), Data Flow (where do inputs come from, where do returns go), and Side Effects (what DB rows, caches, or globals are mutated). A single combined prompt tends to produce a shallow answer across all three.
- **Force the AI to cite line numbers:** Require the AI to point to the specific snippet that justifies each claim in its map. This turns "trust me" output into something you can jump to and verify in seconds. If it can't cite a line, it inferred rather than traced.
- **Trace transitively, but stop it from over-trusting its trace:** For deep call chains, explicitly ask the AI to go 2–3 levels deep and flag where it stopped tracing rather than silently truncating. Otherwise, a dependency four calls deep gets dropped with no warning.

Using AI to surface implicit dependencies

Explicit dependencies are easy to find if one file imports another, any tool can show you that instantly. The harder problem is dependencies that don't show up as a connection between files at all, because they work through shared state, timing, or assumptions nobody wrote down. This is where AI is genuinely useful if you point it at the right categories.

Here's what to look for, in plain terms:

1. Configuration and environment variables
The function you're migrating may look clean, but something it calls maybe several layers deep might quietly depend on a config value or environment variable. If you don't trace that far down, you can migrate the top-level function and lose a behavior nobody documented.

2. Shared mutable state
Any variable that lives outside the function itself, a module-level variable, a cache, a class attribute shared across instances can silently connect two functions that never call each other directly. One function writes to it, another reads from it, and neither knows about the other.

3. Time and ordering dependencies
Some code only works because of an assumption about when it runs or what ran before it, an assumption that was never enforced in code, just true by coincidence of how things are currently called. Change the order during a migration, and this kind of dependency breaks without warning.

4. Implicit contracts with callers
A function's behavior is a promise, even when that promise isn't written anywhere. If most callers handle a certain return value correctly but one doesn't, that one caller is quietly relying on something never happening in practice. Changing the function's behavior can break that caller.

5. External system coupling
Hardcoded timeouts, retry limits, or other fixed values often exist because of hard-won experience with how an external system behaves, not because the number was arbitrary. These reasons are almost never documented, so it's easy to "clean up" a number during migration and reintroduce a problem that was already solved once.

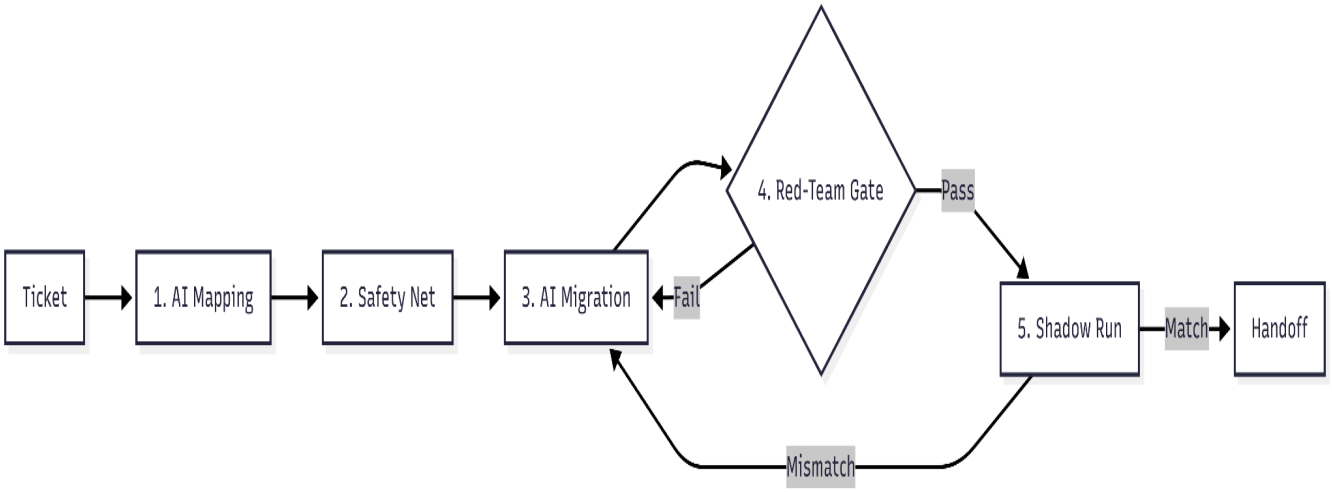
Cross-checking AI-generated dependency maps against the actual code, treating the map as a hypothesis, not ground truth

This is the section where the "human-led red-teaming" theme should first appear, in short treat it as a preview of the mindset the whole back half of the module will apply at scale.

An AI-generated dependency map will typically fail in a few predictable ways, and learners should go looking for these specifically rather than doing a generic read-through:

- **False negatives from truncated tracing** — the AI stopped following a call chain before reaching the actual side effect, especially through indirection (dynamic dispatch, reflection, string-based routing, decorators).
- **False positives from pattern-matching** — the AI infers a dependency because the shape of the code resembles a known pattern, not because it actually traced the specific path (e.g., assuming a function talks to a database because its name contains "save," when it actually only mutates an in-memory object).
- **Confidentially stated omissions** — dependencies the AI simply didn't mention, with no indication of uncertainty, which is more dangerous than an explicit "I'm not sure about this" because it reads as complete.

The Verification Technique: Pick a sample of claims from the AI's map, say, a third of them, heavily weighted toward side effects or hidden dependencies and manually trace those specific claims against the source code. If the sample holds up, you can spot-check the rest. If you find even one fabricated or missed dependency in the sample, treat the whole map as unreliable and verify it method by method.



Building a Safety Net Before Changing Anything

Before you change a single line of legacy code, you need a way to know for certain, not by feeling whether your change broke anything.

Auditing Existing Test Coverage

Start by being honest about what tests actually exist, not what you assume exists. Legacy modules often have some tests, but they're frequently outdated, testing the wrong thing, or quietly skipped in CI without anyone noticing. Don't trust a green test suite until you've looked at what it's actually checking.

Ask yourself three questions before writing anything new:

- Do tests exist for this module at all, and are they actually running?
- Do they test real behavior, or just that the code doesn't crash?
- Do they cover the edge cases, or only the happy path?

Most of the time with legacy code, the honest answer is "coverage is thin or misleading." That's fine, it's the normal starting point. The point of this audit isn't to judge the past, it's to know exactly how much you can and can't rely on before you start.

Characterization / Golden-Master Testing

If proper tests don't exist, you can't wait for someone to write a full spec before you start migrating legacy code often doesn't have one, and never will. So instead of testing what the code should do, you test what it actually does right now. This is called characterization testing (also called golden-master testing).

The idea is simple: run the current code with a range of realistic inputs, record exactly what comes out, and save that as your baseline. You're not judging whether the output is correct, you're just locking it in as "this is what happens today." Later, after migration, you run the same inputs through the new code and compare. Any difference is a signal, not proof of a bug but it's something you now have to explain, not something that slips through unnoticed.

This matters because it flips the whole safety net from opinion to evidence. Instead of "I think this still works," you get "this input produced this exact output before, and it produces the same output now." That's a much stronger claim, and it's one you can automate and rerun as many times as you want.

Using AI to Help Draft Characterization Tests

Writing a wide net of characterization tests by hand is slow and easy to under-cover. This is a good place to let AI do heavy lifting but the division of labor matters.

AI is good at generating a large number of test cases quickly once you show it the function and its current behavior: normal inputs, boundary values, empty inputs, unusual types, unexpected combinations. It can also help you notice input combinations you wouldn't have thought to test yourself

Identifying seams

A seam is a point in the code where you can change behavior without editing that exact spot, usually implemented as a feature flag that decides, at runtime, whether a call goes through the old path or the new one. Function boundaries, class boundaries, and dependency injection points are the natural places to put these flags, since they're already defined points where one piece of code calls into another.

Seams matter for two reasons. First, they let you test one path in isolation without dragging in everything around it. Second, patterns like Strangler Fig and branch by abstraction depend on them. You need a flag to send some traffic to new code while keeping the old path live as a fallback. Without a seam, there's no way to run old and new logic side by side; you're forced into an all-or-nothing switch.

If a module has no natural seams, that's useful to know too; it means the migration is riskier, and adding a flag point around the module should be treated as its own small, verified first step before touching the real logic.

AI-Assisted Migration and Optimization Strategies

Once the safety net is in place, the next decision is how to migrate. In real production systems, almost every one of these strategies is actually implemented through a feature flag, a flag service checked at runtime to decide which code path runs. The strategy names below describe how you use that flag, not four unrelated techniques.

Big Bang vs. Incremental

A big bang migration ships without a flag at all. The old code is deleted, the new code takes over, and there's no runtime switch to fall back on if something's wrong. Rolling back means a full revert and redeploy, not a config change.

An incremental migration keeps a flag in place around the change, so old and new code both exist in production at the same time, and which one runs is controlled by config, not by a deploy. If something breaks, you flip the flag back, no rollback deploy needed, no re-triggering CI/CD, just a config change that takes effect in seconds.

For legacy code with weak documentation and thin tests, incremental i.e., flag-guarded is almost always the safer default, because your rollback is instant instead of a full redeploy.

Strangler Fig Pattern

This is usually built with a **percentage rollout flag**. You wrap the call site with a check like "if flag enabled for this user/request, call new implementation; else call old" and instead of turning it fully on, you ramp the percentage: 1% of traffic, then 5%, then 25%, then 100%, watching error rates and metrics at each step.

The flag is usually scoped to something meaningful: a specific customer segment, a specific endpoint, a specific region so you can strangle one use case at a time rather than one giant on/off switch. The old code path stays in the codebase, reachable by the same flag, until the new path has been at 100% long enough that the team is confident in removing the old code entirely.

Branch by Abstraction

Here the flag doesn't sit at the call site, it sits **inside the abstraction/interface layer** itself. All calling code goes through the interface without knowing a flag exists at all; the interface implementation is what checks the flag and decides whether to delegate to the old class or the new one.

This is the pattern to use when callers are numerous and scattered across the codebase; you don't want to wrap every call site in a flag check individually. Instead, one flag, checked in one place (the abstraction), controls the behavior for every caller simultaneously. When you're ready, you flip that single flag, and nothing else in the codebase needs to change.

Parallel Run / Shadow Comparison

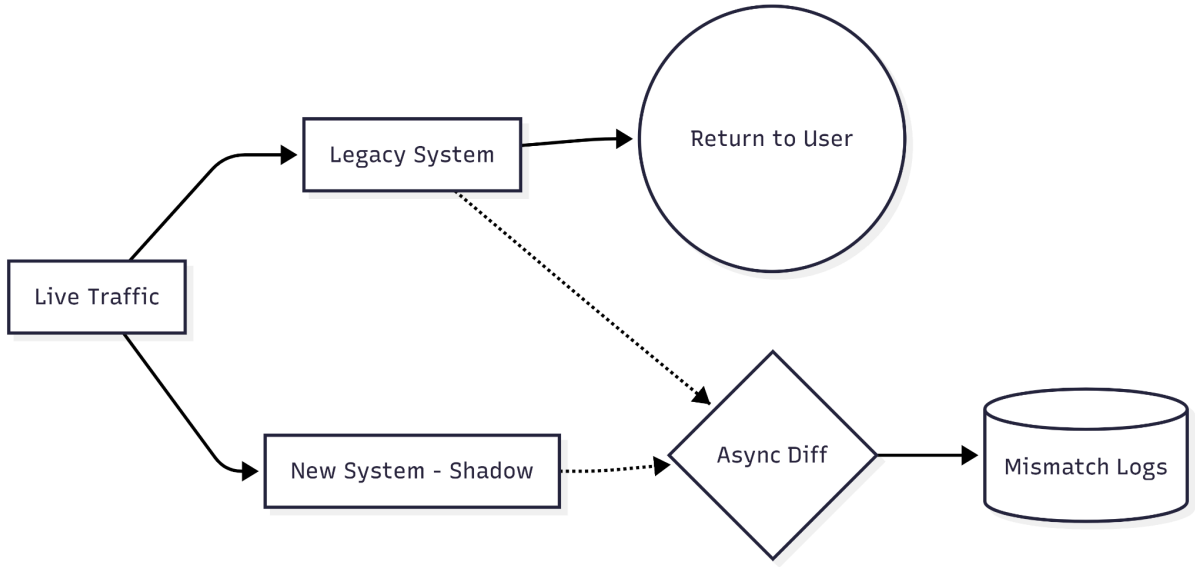
Here the flag controls **whether the new code runs at all, not whether its output is used**. A common real-world setup: the flag is "on" for shadow mode, meaning every request still gets its real response from the old code, but the new code also runs in the background on the same input, and its output gets logged and diffed against the old output silently, with no risk to the user, since the new code's result is never returned or acted on.

Teams typically run this with a separate flag from the rollout flag; one flag says "run new code in shadow," a second, later flag says "actually serve new code's output to users." Keeping these as two separate flags is deliberate: it lets you validate correctness (shadow) completely separately from cutting over real traffic (rollout), so a bug in shadow mode literally cannot affect a real user.

Where AI Speeds This Up and Where It Shouldn't Be Trusted

AI is genuinely useful for the mechanical, repetitive parts of wiring up flags: writing the flag-check boilerplate at call sites, generating the interface/abstraction wrapper code, scaffolding the logging and diffing code for a shadow comparison, and translating old syntax to new syntax inside the branch being flagged. This is well-understood, repetitive work AI does quickly and with fewer typos than doing it by hand for the tenth time.

What AI should not be trusted to decide unsupervised: **what percentage to roll out to and how fast to ramp it, which flag scope to use (user? region? endpoint?), or whether a mismatch in a shadow comparison is safe to ignore.** Those are judgment calls based on business risk tolerance and knowledge of what actually matters in this specific system. An AI will confidently suggest a rollout percentage or dismiss a shadow mismatch with the same certainty whether it's right or dangerously wrong. A "99% match rate" in a shadow comparison sounds fine until a human realizes the 1% mismatch is concentrated in your highest-value customers.



Verifying No Regression

By this point you have a safety net (characterization tests) and a migration built behind a flag. This section is about actually proving nothing broke, not assuming it, proving it. A migration is only successful if no one notices it happened.

Diffing Old vs. New Behavior

Relying solely on unit tests generated by the AI is a trap. AI is notorious for writing **tautological tests** that are perfectly designed to pass the exact (and potentially flawed) logic it just generated. The most reliable way to verify no regression is strict output diffing.

Run both the old and new code on the same real inputs, and diff the outputs field by field. Don't just check "did it error or not" check the actual values. The most dangerous regressions are the ones that return successfully but with a subtly wrong result. Replay every input you recorded a baseline for earlier against the new implementation. Any mismatch isn't automatically a bug; it might be an intentional, approved behavior change but it must be explained, not ignored. If you can't explain why an output changed, don't ship that path until you can.

Edge Case and Boundary Condition Checklist

Characterization tests and shadow traffic will catch most things, but real traffic is often biased toward the common case. Rare inputs may not show up in your sample at all just by chance. Therefore, you must deliberately test a checklist of edge cases rather than only relying on what happened to occur:

- **Empty, null, or missing inputs:** Does the old system treat an empty string as a null, and did the AI assume otherwise?
- **Minimum and maximum boundaries:** Zero, negative numbers, or extremely large numbers.
- **Duplicate, repeated, and out-of-order calls:** Does the new system handle idempotency exactly like the old system?
- **Malformed or unexpected input types:** Legacy dates and timezone boundary logic.
- **Concurrent/parallel access:** How shared state behaves under load.
- **Failure paths:** What happens when a downstream dependency times out or errors.

These are exactly the categories most likely to be under-tested by both the original legacy tests and by AI-generated migration code. Treat this checklist as mandatory.

Deciding What NOT to Silently Fix

You will find bugs in the old code while doing this work that's normal and expected. The discipline here is: **don't fix them silently as part of the migration.**

A bug that has existed in production for three years is no longer a bug; it is an implicit API contract. If your migrated code accidentally (or purposefully) "corrects" a quirk in the old behavior, that's a behavior change hiding inside what was supposed to be a behavior-preserving migration. It will show up as a confusing mismatch to whoever finds it later without context.

Instead, log every suspected bug you find, flag it separately, and let the flag-guarded rollout ship the faithful migration first. Fixing the bug becomes its own deliberate, reviewed change decided on its own merits, not smuggled in because it happened to be easy to fix while you were already in that part of the code.

Handing Off to Deployment

You may not own deployment, rollout, or rollback but you're the only one who currently understands the migration in detail. This section is about transferring that understanding, not just the code.

What to document for the deployment team

At minimum, hand off:

- **What changed and why**: a plain-language summary, not just a diff
- **The flag(s) involved**: names, scopes, and current rollout state (shadow only? 5%? 100%?)
- **What was verified**: which characterization tests passed, what the shadow comparison mismatch rate looked like, and over what volume of traffic
- **Known differences from the old behavior**: anything you deliberately changed (bug fixes, cleanups) and why, clearly separated from the faithful migration
- **What wasn't tested**: be honest about gaps, especially around rare edge cases or dependencies you couldn't fully verify

The last point matters more than people think. A deployment team that knows "we didn't get shadow coverage on the refund path" can watch that path carefully after rollout. A deployment team that assumes full coverage won't know to watch anything in particular.

Flagging rollback considerations

Every flag-guarded migration should ship with an explicit rollback plan, not an assumed one:

- Confirm the flag can be flipped back without a redeploy, and confirm who has permission to flip it
- Note whether rollback is instant and fully safe, or whether the new code has already caused side effects (e.g., writes to a new data format) that make rolling back non-trivial past a certain point
- If rollback isn't clean past a certain rollout percentage or a certain point in time, say so explicitly this is exactly the kind of detail that gets missed under pressure during an incident

Writing migration notes and PR descriptions

The PR description should let someone unfamiliar with the migration understand, in under a minute: what changed, why, what pattern was used (Strangler Fig, branch by abstraction, etc.), what flag controls it, and what to check if something looks wrong after merge. Link out to the fuller migration notes for anyone who needs the full detail the PR itself should be a summary, not the whole story.

Red-Teaming the AI-Generated Result

Adopting an Adversarial Mindset

Everything up to this point mapping, safety nets, migration, verification has been about building things carefully. This section flips the posture: your job now is to try to break what you (or the AI) just built, before someone else does it for you in production. You are no longer the author; you are the auditor.

The default assumption should be that the AI-generated migration is wrong somewhere, not that it's probably fine. This isn't cynicism, it's just accurate. AI output reads as confident and clean regardless of whether it's correct, so confidence in the output tells you nothing about its correctness. The only way to know is to go looking for problems directly, rather than skimming for anything that looks obviously off.

Hunting for Common Failure Modes

AI doesn't make syntax errors anymore it makes deep semantic errors that compile perfectly. A few patterns show up repeatedly, and it's worth hunting for each by name rather than relying on a general read-through:

- **Hallucinated APIs**: a method, library function, or config option that doesn't actually exist (e.g., calling .flattenDeep() on an array that only supports .flat()), or exists but doesn't do what the AI assumed.
- **Subtly wrong edge-case logic**: an off-by-one error, a flipped condition, or a boundary handled differently than the original.
- **Security vulnerabilities**: missing input validation, weakened sanitization, or auth checks that existed in the old code but got dropped during the "cleanup" of the rewrite.
- **Outdated patterns**: code that compiles and runs, but uses an approach that's deprecated or already known to be wrong in the current stack.
- **Silent scope creep**: changes to things nobody asked to have changed (like "helpfully" refactoring a nearby database connector), tucked into the same diff as the intended migration.
- **Tautological tests**: AI-written tests that pass by construction (e.g., asserting a mocked value equals itself) without actually verifying real behavior against the legacy baseline.

Deliberately probing boundary conditions and failure paths the AI likely didn't consider

AI tends to write for the case it was shown or the case most represented in its training the common path. It's far less reliable on the paths nobody talks about: what happens on empty input, what happens when a downstream call fails, what happens under concurrent access. These are exactly the paths worth testing by hand, because they're the ones least likely to have been considered carefully during generation.

Verifying the AI's dependency map and migration output against each other for inconsistencies

The dependency map you built earlier and the migration code you have now should agree with each other. If the map says a function reads a particular env variable, check that the migrated code still reads it. If the map claims this module writes to a Redis cache, but the AI's migrated code has no caching logic, you have caught the AI in a contradiction. Treating these two AI-generated artifacts as cross-checks on each other rather than trusting each in isolation is often how you catch a hidden dependency that neither document called out clearly on its own.

A Practical Red-Team Checklist

Don't review AI code organically by just reading top-to-bottom. Use a short, repeatable, structured checklist to run against any AI-generated migration before it ships. Treat this checklist as a gate, not a suggestion nothing moves past this list unresolved.

- **Correctness:** Does the output match the characterization tests exactly? For any mismatch, is there a clear, deliberate reason?
- **Test quality:** Do the tests actually exercise real behavior, or do they pass regardless of what the code does? Do they assert against the golden-master output, or just against the new code's own logic?
- **Security:** Is every validation, sanitization, and auth check from the original present in the new code? Were any new inputs or surfaces introduced without equivalent checks?
- **Dependencies**: Does the migrated code account for every item in the verified dependency map config, env vars, shared state, ordering, external coupling? Did it introduce any unapproved new packages or libraries along the way?
- **Style/convention fit**: Does the code match the codebase's actual conventions, or does it quietly introduce a different pattern because that's what the AI defaulted to?
- **Scope:** Is every change in the diff something that was actually asked for? Anything extra gets pulled out into its own reviewed change.

Prompting Practices That Reduce Bad Output

You can prevent a good portion of red-team failures before they happen by writing better prompts. The AI can only be as careful as the prompt lets it be.

Providing Context and Constraints

Give the AI the actual constraints up front: what must stay identical, what's explicitly out of scope, what the target environment/framework version is, and what it should flag rather than silently decide on its own. A prompt with no constraints gives the AI permission to make every judgment call itself including ones it shouldn't be making.

Weak prompt: "Migrate this Python 2 script to Python 3."

Better prompt: "Migrate this Python 2 script to Python 3. You are strictly forbidden from changing the database driver. Preserve the exact JSON response structure, including the typo in the recieved_date key."

The second version leaves no room for the AI to "helpfully" fix something on its own judgment; it tells the AI exactly where the boundaries are, including a boundary ("preserve the typo") that looks wrong but is deliberate.

Requesting Tests Alongside Code

Ask for tests in the same request as the migration, not as an afterthought and ask for tests that check real behavior against the characterization baseline, not tests generated purely from the AI's own new implementation (which will just confirm the new code agrees with itself). Asking for both together also forces the AI to work through edge cases explicitly before writing the implementation, rather than after.

Using AI-Explained Reasoning as a Red-Team Input, Not Proof

If you ask the AI to explain why it made a particular change, treat that explanation as one more thing to verify not as evidence the change is correct. A confident, well-written explanation for a wrong decision is still wrong; the fluency of the explanation has no bearing on its accuracy.

Use these explanations to find the specific assumption the AI made, then go check whether that assumption actually holds. Treat the explanation as a red-team input, not a conclusion.

Bringing It Together

Migrating legacy code has always been one of the most hazardous tasks in software engineering. AI doesn't remove that hazard; it just changes the shape of the risk. It speeds up the typing, but it cannot safely speed up the thinking.

The Full Loop

To safely migrate undocumented, fragile systems using AI, you must enforce a strict, repeatable loop:

- **Phase 1: AI-Assisted Mapping.** Use constrained, structured prompts to force the AI to map call chains, data flows, and implicit dependencies. Cross-check its map against reality.
- **Phase 2: The Safety Net.** Do not touch the code until you have a Golden Master suite of characterization tests. Use the AI to generate the boilerplate inputs, but never let it decide what the "correct" output is.
- **Phase 3: AI-Assisted Migration.** Use incremental patterns (Strangler Fig, Branch by Abstraction). Let the AI translate the syntax and write the scaffolding.
- **Phase 4: Human-Led Red-Teaming.** Stop being the author and become the auditor. Hunt for tautological tests, hallucinated APIs, and silent scope creep. Strictly diff the old output against the new output.

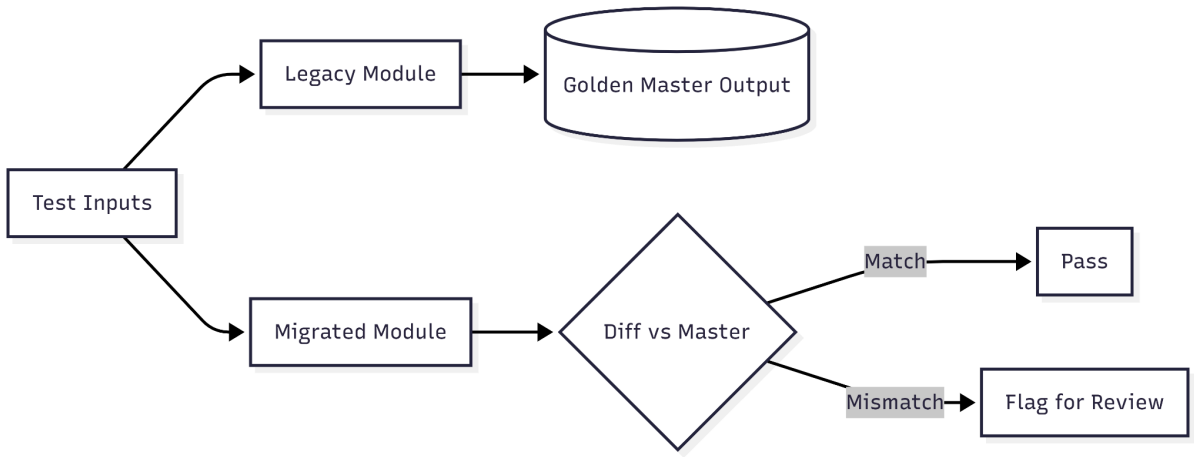


Diagram: Migration lifecycle diagram

Building Your Personal Checklist

Every codebase has its own unique brand of chaos. Take the red-team checklist from Section 8 and adapt it to your specific environment. If your legacy system relies heavily on a bizarre caching layer, add "Explicitly audit cache invalidation" to your mandatory pre-ship gate.

AI is a powerful engine. It is incredibly powerful for clearing the site and moving the heavy dirt, but if you let it drive itself unsupervised, it will knock over the load-bearing walls. Treat it as a high-speed typist, not a senior architect, and your migrations will succeed.

Gem

- Migrating a Legacy Modules Without Regression and critically reviewing AI-Generated Code